

AWS E-Commerce

Data Lakehouse

End-to-End Project

RDS → DMS → S3 → Kinesis → Glue → Iceberg → Athena →
Redshift → QuickSight

Orchestrated by Step Functions · Monitored by CloudWatch

Stat	Value
Phases completed	6 (Phase 0 → Phase 5/6)
AWS services used	10+ native services
Glue ETL jobs	3 (CDC/SCD2, Clickstream, Fact Orders)
Iceberg tables	3 (users_scd2, sessions, fact_orders)
Infrastructure	Terraform (IaC)
Approximate project cost	< \$10 total (demo scale)

Why I Built This

I wanted to understand how data actually moves in a real company — not just querying a CSV or running a notebook, but the full journey from a live database all the way to a dashboard someone can look at. Most tutorials stop at one service. This project was about connecting everything together and seeing what breaks along the way.

The decision to stay AWS-native was deliberate. I could have used open-source tools like Debezium, Kafka, or dbt — and they're great — but every company using AWS eventually gravitates toward managed services because you pay for runtime, not infrastructure management. Learning DMS, Kinesis, Glue, and Step Functions in one project means understanding the trade-offs that engineers actually debate in production.

The real learning wasn't in the happy path. It was in DMS needing a second subnet to even start, Firehose silently dropping records when the Glue schema didn't match, Lake Formation blocking Redshift Spectrum with zero error messages, and SCD2 requiring a two-step MERGE because Iceberg doesn't support updating and inserting in a single pass.

Iceberg was a conscious choice over plain Parquet. Plain Parquet is write-once — you can't update a row, you can't delete without rewriting the file, and you can't query 'what did the table look like last Tuesday.' Iceberg gives you all of that on top of S3, which means you get data warehouse semantics (ACID, time-travel, schema evolution) without paying data warehouse prices.

Architecture Overview

The project is split into two data ingestion tracks running in parallel, both landing in S3 and being processed by Glue into Iceberg tables.

Track	Source	Transport	Landing Zone	Processing
Transactional (CDC)	RDS PostgreSQL	AWS DMS (log-based CDC)	S3 raw/cdc/	Glue SCD2 job
Behavioral (Clickstream)	Python producer	Kinesis + Firehose	S3 raw/clickstream/	Glue sessionization job

After processing, the three Iceberg tables (users_scd2, sessions, fact_orders) are queryable via Athena directly, via Redshift Spectrum for BI workloads, and visualized in QuickSight dashboards. Step Functions orchestrates the three Glue jobs in the right sequence, and CloudWatch alarms alert on failures.

S3 Bucket Layout

```
s3://varun-lakehouse-a86de32c/
  raw/
    cdc/public/users/      ← DMS CSV output (LOAD + delta files)
    cdc/public/orders/    ← DMS CSV output
    clickstream/          ← Firehose GZIP JSON
  curated/
    users_scd2/            ← Iceberg table
    sessions/             ← Iceberg table
    fact_orders/          ← Iceberg table
```

Phase 0: Infrastructure + Mock Data

Terraform · RDS PostgreSQL · Python data generator

What this phase does

Everything starts here. Terraform provisions the entire AWS environment from a single terraform apply — VPC, subnets, security groups, RDS instance, S3 bucket, and the IAM roles needed for later phases. This means the whole infrastructure is version-controlled and reproducible.

Once RDS is up, a Python script generates realistic fake e-commerce data: users with addresses, orders with amounts and statuses, and a mix of active/inactive accounts. This synthetic dataset is what DMS will watch and replicate.

Key decisions

- Terraform over console clicks — so the setup is repeatable and documentable. One terraform destroy tears everything down cleanly.
- RDS logical replication enabled via parameter group — this is a prerequisite for DMS CDC. Without `rds.logical_replication = 1`, DMS can only do full-load, not change capture.
- Separate subnet group created for RDS — DMS requires at least two subnets in different availability zones. Skipping this causes DMS to fail silently at task creation.
- Public subnet for simplicity — in production you'd use private subnets with VPC endpoints, but for a learning project public access with IP-restricted security groups is the right call.

Output after this phase

Resource	Details
S3 bucket	varun-lakehouse-a86de32c
RDS endpoint	PostgreSQL 15, db.t3.micro
Tables created	users (1,000 rows), orders (2,000+ rows)
Terraform state	Stored locally, all resources tagged

Phase 1: CDC with AWS DMS

Database Migration Service · Change Data Capture · S3 as target

What this phase does

DMS connects to the RDS PostgreSQL instance as a replication client and watches the write-ahead log (WAL). Every INSERT, UPDATE, and DELETE that happens in the database gets captured and written to S3 as a CSV file. The first file is a full load (LOAD00000001.csv) containing all existing rows. Every subsequent file contains only the changes, with an Op column indicating I (insert), U (update), or D (delete).

DMS is not cheap to leave running — a `dms.t3.micro` replication instance costs around \$0.048/hr. The key gotcha here was that DMS needs the replication instance in the same VPC as RDS, and it needs an internet gateway to reach S3. Without the S3 VPC endpoint (or the internet gateway), the task sits in 'Starting' state forever with no error.

The CDC output format

DMS writes CSV files where the first column is the operation type for delta files, but the LOAD file has no Op column. This asymmetry is handled in the Glue job by reading both files differently and adding a synthetic Op = 'I' to the load file before combining them with `unionByName`.

What the S3 output looks like

```
raw/cdc/public/users/LOAD00000001.csv    ← all existing rows, no Op column
raw/cdc/public/users/20260429-142736970.csv ← delta: I/U/D rows
```

Key decisions

- CSV over Parquet for DMS output — DMS can write Parquet but it requires additional configuration and the column mapping needs to be exact. CSV is simpler for a first setup and the Glue job handles the type casting anyway.
- Task type: Full load + CDC — this does the initial snapshot first, then switches to ongoing replication automatically.
- IAM role for DMS — let AWS auto-create the `dms-vpc-role` on first console use instead of managing it in Terraform. Terraform managing it causes conflicts if it already exists from a previous DMS setup.

Phase 2: Real-Time Clickstream

Kinesis Data Streams · Kinesis Firehose · GZIP JSON in S3

What this phase does

A Python producer script generates clickstream events (views, clicks, add-to-carts, purchases) and sends them to a Kinesis Data Stream. Kinesis Firehose picks them up, buffers for 60 seconds or 5MB, compresses with GZIP, and lands them in S3 with a date-partitioned prefix.

This simulates what a real e-commerce site does — every user action on the website fires an event that gets streamed in real time. The batch CDC track captures what happened in the database. The stream track captures how users behaved on the site.

The Firehose gotcha

Firehose format conversion to Parquet requires the Glue table schema to exist BEFORE the delivery stream is created, and the column names and types must match the JSON exactly. One mismatch causes Firehose to silently drop records to the error prefix with no warning in the console. The fix was to skip Parquet conversion and use GZIP JSON instead — Spark reads GZIP natively, so the Glue job works fine with it.

Event schema

Field	Type	Example
event_id	string (UUID)	a3f2c1d4-...
user_id	string	user_042
event_type	string	view / click / add_to_cart / purchase
product_id	string	prod_118
timestamp	string (ISO)	2026-04-29T14:27:36Z
session_id	string	sess_8821
device	string	mobile / desktop
referrer	string	google / direct / instagram

Cost note

Kinesis Data Stream costs \$0.015/hr per shard even when idle. The stream was deleted immediately after validating files appeared in S3. Total cost for this phase was under \$0.50.

Phase 3 & 4: Glue ETL Jobs + Iceberg Tables

PySpark · Apache Iceberg · SCD Type 2 · Sessionization · Fact enrichment

Overview

Three Glue jobs transform raw data into curated Iceberg tables. They run in sequence because Job 3 depends on the output of Job 1. Step Functions enforces this order.

Job	Input	Output	Pattern
cdc-scd2-users-job	S3 raw/cdc/users/ (CSV)	users_scd2 (Iceberg)	SCD Type 2 with MERGE
clickstream_process	S3 raw/clickstream/ (GZIP JSON)	sessions (Iceberg)	Dedup + sessionization
fact-order-job	S3 raw/cdc/orders/ + users_scd2	fact_orders (Iceberg)	CDC dedup + user enrichment

Job 1: CDC + SCD Type 2 (cdc-scd2-users-job.py)

SCD2 (Slowly Changing Dimension Type 2) means instead of overwriting a user record when something changes, you close the old version and insert a new one. This gives you a full history of what every user's profile looked like at any point in time.

The schema has `effective_date`, `expiration_date`, and `is_current`. When a user updates their address, the old row gets `expiration_date = now()` and `is_current = false`. A new row is inserted with the new address, `effective_date = now()`, and `is_current = true`.

Why two steps instead of one MERGE? Iceberg's MERGE INTO can match and update OR insert, but it can't do both for the same matched row in a single statement. Closing the old record is an UPDATE. Inserting the new version is an INSERT. They have to be separate operations.

Deletes are handled as soft deletes — the row is closed (`is_current = false`) rather than physically removed. This preserves the historical record for auditing and means fact tables that reference `users_scd2` still work even for deleted users.

Job 2: Clickstream Sessionization (clickstream_process.py)

Raw clickstream has one row per event. The Glue job deduplicates on `event_id` (Firehose can deliver the same event twice), casts the timestamp, and then aggregates by (`user_id`,

session_id, session_hour, device, referrer) to produce session-level metrics: event_count, clicks, add_to_carts, purchases, views.

session_hour is derived by truncating the timestamp to the hour using date_trunc. This makes the sessions table partitioned by day in Iceberg, which means Athena scans less data when you query a date range.

Job 3: Fact Orders + User Enrichment (fact-order-job.py)

Orders from the CDC files are deduplicated by keeping the latest version per order_id using a window function (row_number over order_date desc). Deleted orders (Op = 'D') are filtered out. Then the orders are joined to the current snapshot of users_scd2 (is_current = true) to pull in user_name, user_email, and user_status.

The join is left — orders without a matching user still appear in fact_orders, which is the correct behavior. An inner join would silently drop orders if a user was deleted from the source.

user_status was renamed to user_status on the users side of the join to avoid column name ambiguity — orders also has a status column. Without the alias, Spark throws an analysis exception. Small detail, easy to miss.

Phase 5: Analytics Layer

Athena Views · Redshift Spectrum · QuickSight Dashboards

Athena Views

Three views are created on top of the Iceberg tables in Athena. Views don't store data — they're just saved queries that you can reference by name. This means the underlying Iceberg tables can evolve (add columns, partition changes) without breaking the dashboards as long as the view is updated.

View	Purpose	Key metric
daily_revenue	Orders by date and status	revenue, avg_order_value
user_churn	Recently inactive users (last 7 days)	user_name, user_email, status_changed_at
conversion_funnel	Referrer → purchase conversion	conversion_rate (purchases / views)

Redshift Spectrum

Redshift Spectrum lets you run Redshift SQL queries directly against S3 files (including Iceberg tables) without loading data into Redshift. You create an external schema pointing to the Glue Data Catalog, and then Redshift can see all three Iceberg tables.

Lake Formation is the silent blocker here. Even with the IAM role having all the right S3 and Glue permissions, Spectrum returns 0 rows if Lake Formation hasn't explicitly granted `SELECT` on those tables to the `RedshiftSpectrumRole`. The error message gives you nothing — just empty results.

A materialized view (`mv_daily_revenue`) was created inside Redshift for dashboard use — Redshift caches the aggregated result so the dashboard query is fast, and `REFRESH MATERIALIZED VIEW` updates it when needed.

QuickSight Dashboards

Three dashboards connect directly to Athena using Direct Query mode (no SPICE, avoids extra cost). The dashboards cover daily revenue trends, conversion funnel by referrer, and user churn KPIs. Each uses the Athena views as the data source so the underlying logic stays in SQL, not in QuickSight calculated fields.

Phase 6: Orchestration + Monitoring

Step Functions · Glue Data Quality · CloudWatch Alarms · SNS

Step Functions Orchestration

The three Glue jobs need to run in order: SCD2 first (because fact_orders reads from users_scd2), then clickstream, then fact orders. Step Functions enforces this with a simple linear state machine using startJobRun.sync — it waits for each Glue job to complete before starting the next one.

Each state has a Catch block pointing to a Pipeline_Failed terminal state. If any job fails, execution stops there and CloudWatch captures the failure. Without Catch, a failed Glue job would leave Step Functions in a running state until it times out.

State machine flow

```
Process_CDC (scd-type2-gluejob)
  ↓ success
Process_Clickstream (clickstream_process)
  ↓ success
Process_Facts (fact-order-job)
  ↓ success          ↓ any failure
Pipeline_Success    Pipeline_Failed
```

Glue Data Quality

A ruleset was created on fact_orders with 5 rules: order_id completeness >= 99%, order_id uniqueness >= 99%, user_id completeness >= 99%, column count >= 7, and row count >= 400. To test the alarm, the row count rule was temporarily raised to >= 500 (actual count was ~450), which triggered the CloudWatch alarm and sent an SNS email notification.

CloudWatch Alarms

Alarm	Metric	Condition	Action
lakehouse-pipeline-failure	StepFunctions ExecutionsFailed	> 0	SNS email
lakehouse-data-quality-alarm	Glue DQ rules failed	> 0	SNS email

The data quality alarm went through three states during testing: INSUFFICIENT_DATA (right after creation, before any run), IN_ALARM (after triggering the intentional failure),

and OK (after restoring the correct threshold). This is exactly how production monitoring works — you want to see the alarm actually fire before trusting it.

Why Iceberg Over Plain Parquet

This is worth explaining in detail because the choice of Iceberg shapes the entire storage layer of the project.

Capability	Plain Parquet	Apache Iceberg
Update a row	Not possible — must rewrite the file	Supported (copy-on-write)
Delete a row	Not possible without full rewrite	Supported
Query at a point in time	Not possible	Time travel (SELECT AS OF)
Schema evolution	Manual file management	ALTER TABLE ADD COLUMN
ACID guarantees	None	Full ACID
Partition evolution	Requires data migration	Change partition without rewriting
Concurrent writers	Unsafe	Optimistic concurrency control

For this project specifically, Iceberg made SCD2 possible. Without UPDATE support, you can't close old records. You'd have to read the entire table, modify rows in memory, and write it all back — which doesn't scale and isn't atomic.

The warehouse path is in the curated/ prefix of the same S3 bucket. The Glue Data Catalog stores the metadata (schema, partition info, snapshot history) and Athena/Redshift/Spark all use it as the catalog entry point.

Total Project Cost

Phase	Main cost drivers	Approx. cost
Phase 0: Infrastructure	RDS t3.micro (~\$0.02/hr), S3	~\$0.50
Phase 1: DMS	dms.t3.micro (\$0.048/hr), task runtime	~\$0.50
Phase 2: Kinesis	\$0.015/hr per shard (deleted after test)	~\$0.40
Phase 3/4: Glue jobs	\$0.44 per DPU-hour, 3 jobs × 2 runs	~\$2.00
Phase 5: Analytics	Athena (\$0.01), Redshift 30 min (\$1.50)	~\$1.60
Phase 6: Orchestration	Step Functions, CloudWatch, SNS	~\$0.50
Total		< \$6

The biggest cost lever is Redshift Serverless — it starts billing at \$2.88/hr at 8 RPUs minimum. The workgroup was stopped immediately after taking screenshots. Everything else in the project is either free-tier eligible or cents-per-run.

What Production Would Look Like

This project is intentionally demo-scale, but the patterns map directly to production. Here's what would change:

Component	Demo setup	Production upgrade
DMS	Single-AZ replication instance	Multi-AZ, larger instance class
Kinesis	1 shard, manual producer	Auto-scaling shards, real application SDK
Glue jobs	Manual trigger via Step Functions	EventBridge schedule (e.g. every hour)
Iceberg tables	Single S3 bucket	Separate buckets per environment (dev/prod)
Redshift	Serverless, stopped when idle	Always-on cluster with reserved pricing
Monitoring	2 CloudWatch alarms	Full CloudWatch dashboard + PagerDuty integration
Security	Public RDS with IP restriction	Private subnets, VPC endpoints, secrets in Secrets Manager
Data quality	Manual ruleset run	Integrated into Step Functions state machine

Built on AWS · Apache Iceberg · PySpark · Terraform